Cloud-Native Database Systems and Unikernels: Reimagining OS Abstractions for Modern Hardware

Viktor Leis
Technische Universität München
leis@in.tum.de

Christian Dietrich
Technische Universität Braunschweig
dietrich@ibr.cs.tu-bs.de

ABSTRACT

This paper explores the intersection of operating systems and database systems, focusing on the potential of specialized kernels for cloud-native database systems. Although the idea of custom, DBMS-optimized OS kernels is old, it is largely unrealized due to the demands of hardware compatibility and the reluctance of users to install specialized operating systems. However, the cloud and the database-as-a-service model make custom OS kernels realistic for the first time. Among specialized OS kernel architectures, unikernels stand out for relying on a single address space, eliminating the need for costly process isolation that is provided by general-purpose operating systems. They offer benefits such as the elimination of system call overhead, direct access to hardware, and reduced complexity. Beyond these immediate advantages, unikernels offer a unique opportunity: the possibility to revisit dated POSIX APIs. By allowing direct interaction with modern hardware primitives, unikernels pave the way for the development of novel abstractions that are not confined to the limitations of older APIs, opening doors to a new era of co-designed, high-performance cloud-native data processing systems and OS kernels.

PVLDB Reference Format:
Viktor Leis and Christian Dietrich. Cloud-Native Database Systems and Unikernels: Reimagining OS Abstractions for Modern Hardware. PVLDB, 17(8): 2115 - 2122, 2024.
doi:10.14778/3659437.3659462

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://doi.org/10.5281/zenodo.10672474.

1 INTRODUCTION

Operating Systems and Database Systems. In an OSDI 2021 keynote [50], Timothy Roscoe defined operating systems "software that multiplexes [a] machine's hardware resources, abstracts the hardware platform, and protects software principals from each other using the hardware". In essence, operating systems provide a uniform, abstract interface to the hardware and isolate different processes accessing the hardware. However, hardware abstraction and process isolation come at a cost, particularly for high-performance database systems that require fine-grained control over the hardware to optimize performance and resource utilization. As a result, it has long been recognized [21, 54] that custom OS kernels could offer more suitable OS APIs tailored to the needs of database systems and therefore better performance.

Why Have Custom OS-Kernels Not Been Successful? In practice, DBMS-optimized OS kernels never gained widespread adoption for two main reasons. First, few users would buy a DBMS that requires installing a custom OS as a prerequisite. Second, since users have different hardware setups, such a custom OS would have to implement and maintain drivers for a myriad of devices. Therefore, outside of narrow niche use cases, database systems simply had to adapt to the restrictions of general-purpose operating systems.

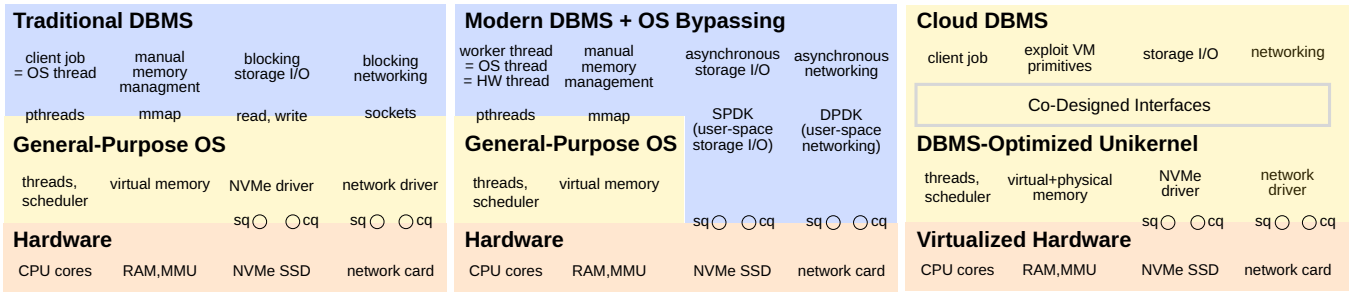
Why This Time Is Different: Cloud. The transition to the cloud has made the use of custom OS kernels in database systems more realistic. Instead of shipping DBMS software to customers for on-premise installation, database systems are increasingly offered as hosted services in public clouds. This shift means that custom kernels only need to be installed by the DBMS vendor, allowing users to benefit from a custom kernel without the burden of administration. Moreover, public clouds typically require support for only a handful of hardware devices. For example, AWS EC2 supports booting custom OS kernels, and a DBMS running on any recent EC2 instance only needs to implement two drivers: an AWS-specific EFA driver for networking and a generic NVMe driver for storage. This makes custom kernels a realistic option for cloud-native DBMSs.

OS Architecture Continuum. OS architectures vary in their degree of isolation: Microkernels [37] maximize isolation by segregating OS components (e.g., file systems) into separate processes. Monoliths, e.g., Linux or Windows, only distinguish between user and kernel space, executing bug-prone drivers [10, 48] with supervisor privileges. Container-based virtualization [40, 41] also falls into the monolith category, as it relies on namespace-isolated heavy-weight processes. Library OSes [2, 19] and the cloud-targeted *unikernels* [29, 31, 38], which are the focus of this work, co-locate all components into one address space without privilege isolation.

Kernel Integration. We argue that unikernels are an ideal foundation for cloud-native database systems as their lack of isolation allows for deep *DBMS/kernel integration*. With a unikernel, any thread can directly access any kernel data structure and hardware device. Hypervisor-based virtualization, where each VM runs a single service, ensures tenant isolation in the cloud. For many cloud services, the overhead of running a general-purpose OS, like Linux, is redundant, as there are no other processes or users to isolate from. Unikernels, therefore, eliminate this unnecessary layer of isolation. Section 3.1 discusses security implications in more detail.

Unikernel Benefits. For high-performance database management systems, unikernels offer several compelling advantages. (i) There is no (cost for) privilege-level transitions. (ii) Applications can directly control hardware *and* manipulate virtual-memory page tables. (iii)

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 8 ISSN 2150-8097.
doi:10.14778/3659437.3659462



(a) Traditional division of labor (b) Modern DBMS with (optional) bypassing (c) Our vision for DBMS/OS co-design

Figure 1: Models of DBMS/OS interaction

Isolation is a major source of OS implementation complexity; hence, unikernels can be smaller, simpler, and more predictable. (iv) Due to their simplicity, unikernels boot more quickly than general-purpose kernels.

New Abstractions. Although the benefits listed above are significant, we believe the most profound advantage of unikernels lies in their ability to move beyond the decades-old POSIX API. Existing OS APIs severely limit the design space: for example, using `mmap` it is not possible to implement ARIES-style write-ahead logging with in-place writes [11]. Unikernels make it both straightforward and efficient to develop new abstractions based on the primitives provided by modern hardware such as the page tables, TLB, and inter-processor interrupts. For example, the virtual-memory page table becomes a simple radix data structure that can be manipulated directly instead of having to rely on expensive and functionally limited system calls. The co-design of data processing systems and OS kernels offers the unique opportunity to develop new abstractions without being restricted to APIs that were designed for single-core systems with scarce memory.

2 OPERATING SYSTEMS AND DATABASES

Before we present specific ideas enabled by co-designing database systems and unikernels in Section 3, let us first discuss why the current state of operating systems falls short for high-performance data processing systems.

Traditional Abstractions. A main task of an operating system kernel is to manage hardware. Specifically, an OS provides a uniform abstraction over hardware and multiplexes it among different users. The four main hardware resources most relevant for database systems are CPU cores, main memory, storage, and networking. In the traditional model, the OS provides the abstraction *threads of execution*, which the OS scheduler dynamically maps to CPU cores. Storage and network I/O are done through synchronous, blocking system calls. When a thread blocks on an I/O operation, it is not scheduled until the device signals completion via an interrupt. In this model, the OS controls all scheduling decisions and receives all hardware signals.

Traditional Abstraction Worked Well. As Figure 1(a) illustrates, a straightforward way to implement a DBMS on top of the traditional OS abstraction is to have one thread (or process) for each client connection. Each thread listens on a network socket for incoming requests using a blocking system call. If a request arrives, the OS schedules the thread for execution on some CPU core. The thread can then process the query and eventually return the result to the client through another blocking socket system call. During query execution, buffer-pool page misses may result in storage I/O system calls and therefore additional context switches. This model is easy to use, conceptually elegant, and provides a clean separation of concerns between the DBMS and OS. The DBMS relies heavily on the abstractions provided by the OS. Through blocking system calls (and, when needed, preemptive multitasking) the OS is in full control and can ensure that all hardware resources are well utilized. In a world where (disk and network) I/O was relatively slow in comparison to the CPU, the model was efficient due to negligible context switching overhead. Consequently, this synchronous model has historically been very successful, remaining the basis for widely-used systems like PostgreSQL.

Traditional Abstraction On Modern Hardware. Unfortunately, on modern hardware, the synchronous model runs into severe performance problems. Modern storage devices are very fast, which means that the kernel CPU overhead of I/O stack CPU can be substantial[1]. Storage devices are also highly parallel: a single modern SSD, for example, is capable of simultaneously executing on the order of 100 I/O requests and over one million requests per second. Consequently, it is nearly impossible to exploit modern storage devices using blocking I/O system calls [23]. A second, more conceptual, problem with the traditional model is that it does not take intra-query parallelism into account, which is crucial on modern hardware with dozens of CPU cores. Intra-query parallelism is challenging because it breaks the one-to-one mapping between client connections and threads.

Asynchronous I/O and Worker Thread Scheduling. To address the challenges associated with synchronous I/O and intra-query parallelism, modern high-performance database systems therefore schedule I/O requests and CPU cores themselves. To do this, they launch as one worker thread per CPU core [33], rely on asynchronous I/O interfaces such as io_uring [23], and avoid OS services such as software RAID and the file system [22, 46]. In this model, the database system has to decide how many threads to use for each query, which means that it requires a scheduler that ideally considers other currently-running queries and whether they are I/O or CPU-bound. The DBMS also has to manage I/O requests, with the

[1] On Linux, every I/O operation takes on the order of 10,000 CPU cycles.